doi:10.1038/nature14539

Deep learning

Yann LeCun[1,2], Yoshua Bengio[3] & Geoffrey Hinton[4,5]

Deep learning allows computational models that are composed of multiple processing layers to learn representations of data with multiple levels of abstraction. These methods have dramatically improved the state-of-the-art in speech recognition, visual object recognition, object detection and many other domains such as drug discovery and genomics. Deep learning discovers intricate structure in large data sets by using the backpropagation algorithm to indicate how a machine should change its internal parameters that are used to compute the representation in each layer from the representation in the previous layer. Deep convolutional nets have brought about breakthroughs in processing images, video, speech and audio, whereas recurrent nets have shone light on sequential data such as text and speech.

Machine-learning technology powers many aspects of modern society: from web searches to content filtering on social networks to recommendations on e-commerce websites, and it is increasingly present in consumer products such as cameras and smartphones. Machine-learning systems are used to identify objects in images, transcribe speech into text, match news items, posts or products with users' interests, and select relevant results of search. Increasingly, these applications make use of a class of techniques called deep learning.

Conventional machine-learning techniques were limited in their ability to process natural data in their raw form. For decades, constructing a pattern-recognition or machine-learning system required careful engineering and considerable domain expertise to design a feature extractor that transformed the raw data (such as the pixel values of an image) into a suitable internal representation or feature vector from which the learning subsystem, often a classifier, could detect or classify patterns in the input.

Representation learning is a set of methods that allows a machine to be fed with raw data and to automatically discover the representations needed for detection or classification. Deep-learning methods are representation-learning methods with multiple levels of representation, obtained by composing simple but non-linear modules that each transform the representation at one level (starting with the raw input) into a representation at a higher, slightly more abstract level. With the composition of enough such transformations, very complex functions can be learned. For classification tasks, higher layers of representation amplify aspects of the input that are important for discrimination and suppress irrelevant variations. An image, for example, comes in the form of an array of pixel values, and the learned features in the first layer of representation typically represent the presence or absence of edges at particular orientations and locations in the image. The second layer typically detects motifs by spotting particular arrangements of edges, regardless of small variations in the edge positions. The third layer may assemble motifs into larger combinations that correspond to parts of familiar objects, and subsequent layers would detect objects as combinations of these parts. The key aspect of deep learning is that these layers of features are not designed by human engineers: they are learned from data using a general-purpose learning procedure.

Deep learning is making major advances in solving problems that have resisted the best attempts of the artificial intelligence community for many years. It has turned out to be very good at discovering intricate structures in high-dimensional data and is therefore applicable to many domains of science, business and government. In addition to beating records in image recognition[1–4] and speech recognition[5–7], it has beaten other machine-learning techniques at predicting the activity of potential drug molecules[8], analysing particle accelerator data[9,10], reconstructing brain circuits[11], and predicting the effects of mutations in non-coding DNA on gene expression and disease[12,13]. Perhaps more surprisingly, deep learning has produced extremely promising results for various tasks in natural language understanding[14], particularly topic classification, sentiment analysis, question answering[15] and language translation[16,17].

We think that deep learning will have many more successes in the near future because it requires very little engineering by hand, so it can easily take advantage of increases in the amount of available computation and data. New learning algorithms and architectures that are currently being developed for deep neural networks will only accelerate this progress.

Supervised learning

The most common form of machine learning, deep or not, is supervised learning. Imagine that we want to build a system that can classify images as containing, say, a house, a car, a person or a pet. We first collect a large data set of images of houses, cars, people and pets, each labelled with its category. During training, the machine is shown an image and produces an output in the form of a vector of scores, one for each category. We want the desired category to have the highest score of all categories, but this is unlikely to happen before training. We compute an objective function that measures the error (or distance) between the output scores and the desired pattern of scores. The machine then modifies its internal adjustable parameters to reduce this error. These adjustable parameters, often called weights, are real numbers that can be seen as 'knobs' that define the input–output function of the machine. In a typical deep-learning system, there may be hundreds of millions of these adjustable weights, and hundreds of millions of labelled examples with which to train the machine.

To properly adjust the weight vector, the learning algorithm computes a gradient vector that, for each weight, indicates by what amount the error would increase or decrease if the weight were increased by a tiny amount. The weight vector is then adjusted in the opposite direction to the gradient vector.

The objective function, averaged over all the training examples, can

[1]Facebook AI Research, 770 Broadway, New York, New York 10003 USA. [2]New York University, 715 Broadway, New York, New York 10003, USA. [3]Department of Computer Science and Operations Research Université de Montréal, Pavillon André-Aisenstadt, PO Box 6128 Centre-Ville STN Montréal, Quebec H3C 3J7, Canada. [4]Google, 1600 Amphitheatre Parkway, Mountain View, California 94043, USA. [5]Department of Computer Science, University of Toronto, 6 King's College Road, Toronto, Ontario M5S 3G4, Canada.

© 2015 Macmillan Publishers Limited. All rights reserved

be seen as a kind of hilly landscape in the high-dimensional space of weight values. The negative gradient vector indicates the direction of steepest descent in this landscape, taking it closer to a minimum, where the output error is low on average.

In practice, most practitioners use a procedure called stochastic gradient descent (SGD). This consists of showing the input vector for a few examples, computing the outputs and the errors, computing the average gradient for those examples, and adjusting the weights accordingly. The process is repeated for many small sets of examples from the training set until the average of the objective function stops decreasing. It is called stochastic because each small set of examples gives a noisy estimate of the average gradient over all examples. This simple procedure usually finds a good set of weights surprisingly quickly when compared with far more elaborate optimization techniques[18]. After training, the performance of the system is measured on a different set of examples called a test set. This serves to test the generalization ability of the machine — its ability to produce sensible answers on new inputs that it has never seen during training.

Many of the current practical applications of machine learning use linear classifiers on top of hand-engineered features. A two-class linear classifier computes a weighted sum of the feature vector components. If the weighted sum is above a threshold, the input is classified as belonging to a particular category.

Since the 1960s we have known that linear classifiers can only carve their input space into very simple regions, namely half-spaces separated by a hyperplane[19]. But problems such as image and speech recognition require the input–output function to be insensitive to irrelevant variations of the input, such as variations in position, orientation or illumination of an object, or variations in the pitch or accent of speech, while being very sensitive to particular minute variations (for example, the difference between a white wolf and a breed of wolf-like white dog called a Samoyed). At the pixel level, images of two Samoyeds in different poses and in different environments may be very different from each other, whereas two images of a Samoyed and a wolf in the same position and on similar backgrounds may be very similar to each other. A linear classifier, or any other 'shallow' classifier operating on

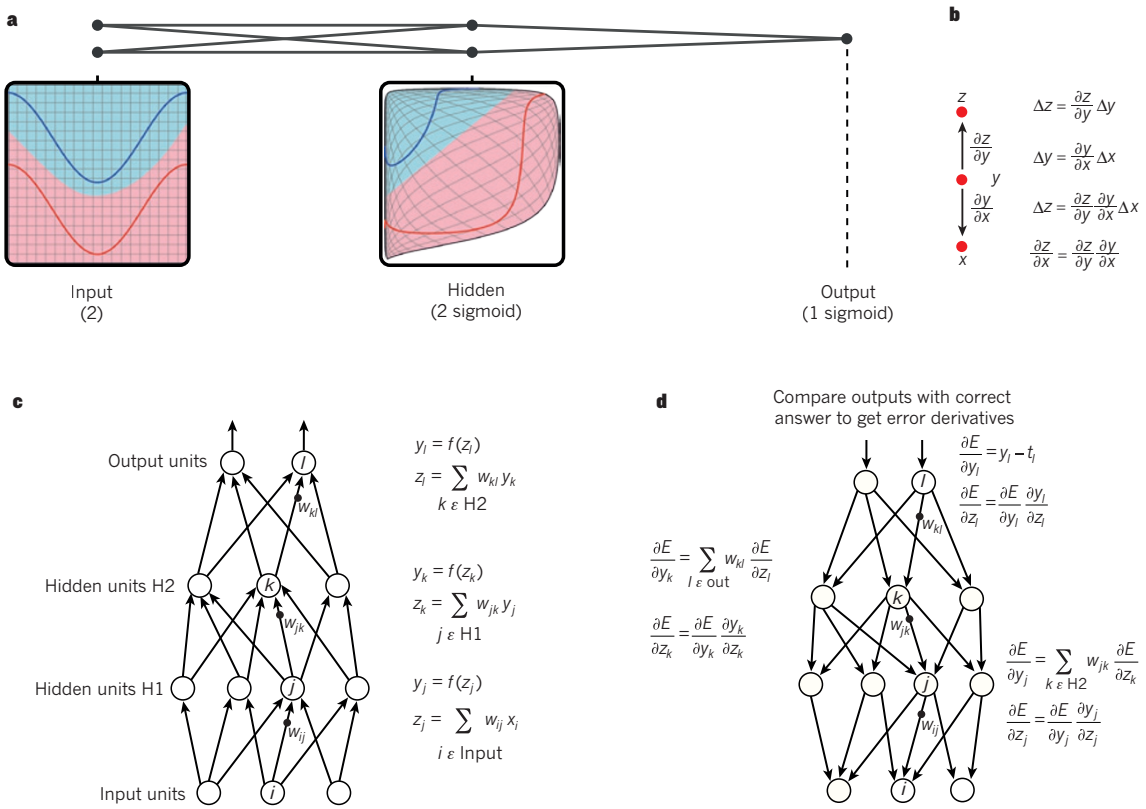

Figure 1 | Multilayer neural networks and backpropagation. **a**, A multilayer neural network (shown by the connected dots) can distort the input space to make the classes of data (examples of which are on the red and blue lines) linearly separable. Note how a regular grid (shown on the left) in input space is also transformed (shown in the middle panel) by hidden units. This is an illustrative example with only two input units, two hidden units and one output unit, but the networks used for object recognition or natural language processing contain tens or hundreds of thousands of units. Reproduced with permission from C. Olah (http://colah.github.io/). **b**, The chain rule of derivatives tells us how two small effects (that of a small change of x on y, and that of y on z) are composed. A small change Δx in x gets transformed first into a small change Δy in y by getting multiplied by $\partial y/\partial x$ (that is, the definition of partial derivative). Similarly, the change Δy creates a change Δz in z. Substituting one equation into the other gives the chain rule of derivatives — how Δx gets turned into Δz through multiplication by the product of $\partial y/\partial x$ and $\partial z/\partial x$. It also works when x, y and z are vectors (and the derivatives are Jacobian matrices). **c**, The equations used for computing the forward pass in a neural net with two hidden layers and one output layer, each constituting a module through which one can backpropagate gradients. At each layer, we first compute the total input z to each unit, which is a weighted sum of the outputs of the units in the layer below. Then a non-linear function $f(.)$ is applied to z to get the output of the unit. For simplicity, we have omitted bias terms. The non-linear functions used in neural networks include the rectified linear unit (ReLU) $f(z) = \max(0,z)$, commonly used in recent years, as well as the more conventional sigmoids, such as the hyberbolic tangent, $f(z) = (\exp(z) - \exp(-z))/(\exp(z) + \exp(-z))$ and logistic function logistic, $f(z) = 1/(1 + \exp(-z))$. **d**, The equations used for computing the backward pass. At each hidden layer we compute the error derivative with respect to the output of each unit, which is a weighted sum of the error derivatives with respect to the total inputs to the units in the layer above. We then convert the error derivative with respect to the output into the error derivative with respect to the input by multiplying it by the gradient of $f(z)$. At the output layer, the error derivative with respect to the output of a unit is computed by differentiating the cost function. This gives $y_l - t_l$ if the cost function for unit l is $0.5(y_l - t_l)^2$, where t_l is the target value. Once the $\partial E/\partial z_k$ is known, the error-derivative for the weight w_{jk} on the connection from unit j in the layer below is just $y_j\,\partial E/\partial z_k$.

© 2015 Macmillan Publishers Limited. All rights reserved

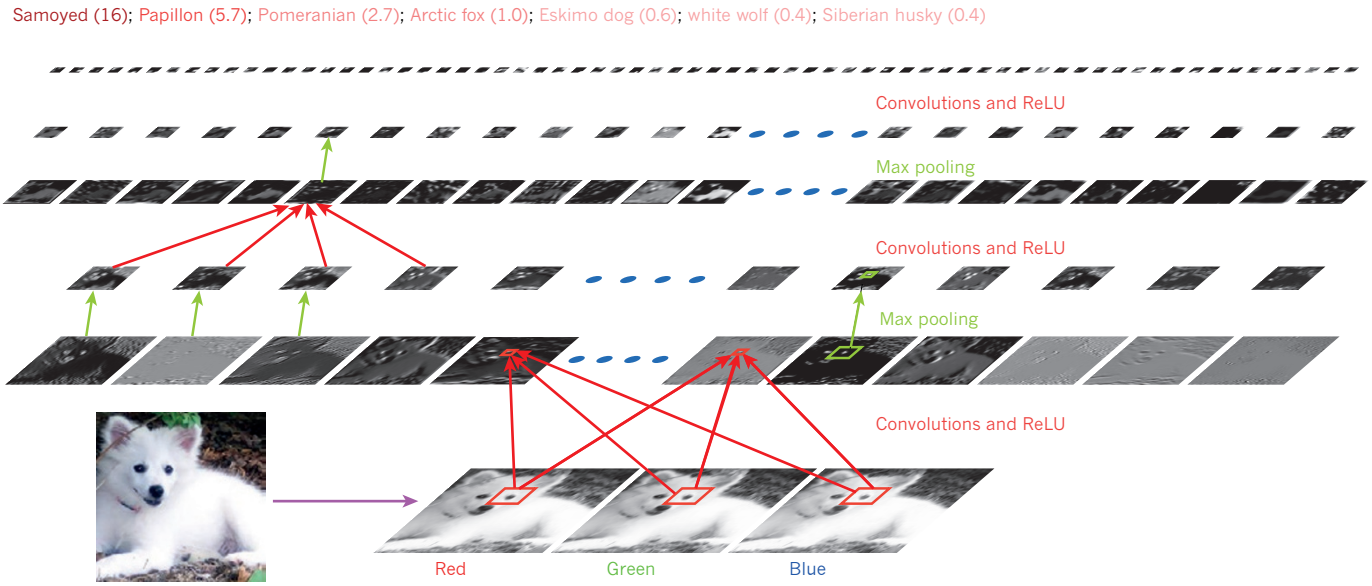

Figure 2 | Inside a convolutional network. The outputs (not the filters) of each layer (horizontally) of a typical convolutional network architecture applied to the image of a Samoyed dog (bottom left; and RGB (red, green, blue) inputs, bottom right). Each rectangular image is a feature map corresponding to the output for one of the learned features, detected at each of the image positions. Information flows bottom up, with lower-level features acting as oriented edge detectors, and a score is computed for each image class in output. ReLU, rectified linear unit.

raw pixels could not possibly distinguish the latter two, while putting the former two in the same category. This is why shallow classifiers require a good feature extractor that solves the selectivity–invariance dilemma — one that produces representations that are selective to the aspects of the image that are important for discrimination, but that are invariant to irrelevant aspects such as the pose of the animal. To make classifiers more powerful, one can use generic non-linear features, as with kernel methods[20], but generic features such as those arising with the Gaussian kernel do not allow the learner to generalize well far from the training examples[21]. The conventional option is to hand design good feature extractors, which requires a considerable amount of engineering skill and domain expertise. But this can all be avoided if good features can be learned automatically using a general-purpose learning procedure. This is the key advantage of deep learning.

A deep-learning architecture is a multilayer stack of simple modules, all (or most) of which are subject to learning, and many of which compute non-linear input–output mappings. Each module in the stack transforms its input to increase both the selectivity and the invariance of the representation. With multiple non-linear layers, say a depth of 5 to 20, a system can implement extremely intricate functions of its inputs that are simultaneously sensitive to minute details — distinguishing Samoyeds from white wolves — and insensitive to large irrelevant variations such as the background, pose, lighting and surrounding objects.

Backpropagation to train multilayer architectures

From the earliest days of pattern recognition[22,23], the aim of researchers has been to replace hand-engineered features with trainable multilayer networks, but despite its simplicity, the solution was not widely understood until the mid 1980s. As it turns out, multilayer architectures can be trained by simple stochastic gradient descent. As long as the modules are relatively smooth functions of their inputs and of their internal weights, one can compute gradients using the backpropagation procedure. The idea that this could be done, and that it worked, was discovered independently by several different groups during the 1970s and 1980s[24–27].

The backpropagation procedure to compute the gradient of an objective function with respect to the weights of a multilayer stack of modules is nothing more than a practical application of the chain rule for derivatives. The key insight is that the derivative (or gradient) of the objective with respect to the input of a module can be computed by working backwards from the gradient with respect to the output of that module (or the input of the subsequent module) (Fig. 1). The backpropagation equation can be applied repeatedly to propagate gradients through all modules, starting from the output at the top (where the network produces its prediction) all the way to the bottom (where the external input is fed). Once these gradients have been computed, it is straightforward to compute the gradients with respect to the weights of each module.

Many applications of deep learning use feedforward neural network architectures (Fig. 1), which learn to map a fixed-size input (for example, an image) to a fixed-size output (for example, a probability for each of several categories). To go from one layer to the next, a set of units compute a weighted sum of their inputs from the previous layer and pass the result through a non-linear function. At present, the most popular non-linear function is the rectified linear unit (ReLU), which is simply the half-wave rectifier $f(z) = \max(z, 0)$. In past decades, neural nets used smoother non-linearities, such as $\tanh(z)$ or $1/(1 + \exp(-z))$, but the ReLU typically learns much faster in networks with many layers, allowing training of a deep supervised network without unsupervised pre-training[28]. Units that are not in the input or output layer are conventionally called hidden units. The hidden layers can be seen as distorting the input in a non-linear way so that categories become linearly separable by the last layer (Fig. 1).

In the late 1990s, neural nets and backpropagation were largely forsaken by the machine-learning community and ignored by the computer-vision and speech-recognition communities. It was widely thought that learning useful, multistage, feature extractors with little prior knowledge was infeasible. In particular, it was commonly thought that simple gradient descent would get trapped in poor local minima — weight configurations for which no small change would reduce the average error.

In practice, poor local minima are rarely a problem with large networks. Regardless of the initial conditions, the system nearly always reaches solutions of very similar quality. Recent theoretical and empirical results strongly suggest that local minima are not a serious issue in general. Instead, the landscape is packed with a combinatorially large number of saddle points where the gradient is zero, and the surface curves up in most dimensions and curves down in the

© 2015 Macmillan Publishers Limited. All rights reserved

remainder[29,30]. The analysis seems to show that saddle points with only a few downward curving directions are present in very large numbers, but almost all of them have very similar values of the objective function. Hence, it does not much matter which of these saddle points the algorithm gets stuck at.

Interest in deep feedforward networks was revived around 2006 (refs 31–34) by a group of researchers brought together by the Canadian Institute for Advanced Research (CIFAR). The researchers introduced unsupervised learning procedures that could create layers of feature detectors without requiring labelled data. The objective in learning each layer of feature detectors was to be able to reconstruct or model the activities of feature detectors (or raw inputs) in the layer below. By 'pre-training' several layers of progressively more complex feature detectors using this reconstruction objective, the weights of a deep network could be initialized to sensible values. A final layer of output units could then be added to the top of the network and the whole deep system could be fine-tuned using standard backpropagation[33–35]. This worked remarkably well for recognizing handwritten digits or for detecting pedestrians, especially when the amount of labelled data was very limited[36].

The first major application of this pre-training approach was in speech recognition, and it was made possible by the advent of fast graphics processing units (GPUs) that were convenient to program[37] and allowed researchers to train networks 10 or 20 times faster. In 2009, the approach was used to map short temporal windows of coefficients extracted from a sound wave to a set of probabilities for the various fragments of speech that might be represented by the frame in the centre of the window. It achieved record-breaking results on a standard speech recognition benchmark that used a small vocabulary[38] and was quickly developed to give record-breaking results on a large vocabulary task[39]. By 2012, versions of the deep net from 2009 were being developed by many of the major speech groups[6] and were already being deployed in Android phones. For smaller data sets, unsupervised pre-training helps to prevent overfitting[40], leading to significantly better generalization when the number of labelled examples is small, or in a transfer setting where we have lots of examples for some 'source' tasks but very few for some 'target' tasks. Once deep learning had been rehabilitated, it turned out that the pre-training stage was only needed for small data sets.

There was, however, one particular type of deep, feedforward network that was much easier to train and generalized much better than networks with full connectivity between adjacent layers. This was the convolutional neural network (ConvNet)[41,42]. It achieved many practical successes during the period when neural networks were out of favour and it has recently been widely adopted by the computer-vision community.

Convolutional neural networks

ConvNets are designed to process data that come in the form of multiple arrays, for example a colour image composed of three 2D arrays containing pixel intensities in the three colour channels. Many data modalities are in the form of multiple arrays: 1D for signals and sequences, including language; 2D for images or audio spectrograms; and 3D for video or volumetric images. There are four key ideas behind ConvNets that take advantage of the properties of natural signals: local connections, shared weights, pooling and the use of many layers.

The architecture of a typical ConvNet (Fig. 2) is structured as a series of stages. The first few stages are composed of two types of layers: convolutional layers and pooling layers. Units in a convolutional layer are organized in feature maps, within which each unit is connected to local patches in the feature maps of the previous layer through a set of weights called a filter bank. The result of this local weighted sum is then passed through a non-linearity such as a ReLU. All units in a feature map share the same filter bank. Different feature maps in a layer use different filter banks. The reason for this architecture is twofold. First, in array data such as images, local groups of values are often highly correlated, forming distinctive local motifs that are easily detected. Second, the local statistics of images and other signals are invariant to location. In other words, if a motif can appear in one part of the image, it could appear anywhere, hence the idea of units at different locations sharing the same weights and detecting the same pattern in different parts of the array. Mathematically, the filtering operation performed by a feature map is a discrete convolution, hence the name.

Although the role of the convolutional layer is to detect local conjunctions of features from the previous layer, the role of the pooling layer is to merge semantically similar features into one. Because the relative positions of the features forming a motif can vary somewhat, reliably detecting the motif can be done by coarse-graining the position of each feature. A typical pooling unit computes the maximum of a local patch of units in one feature map (or in a few feature maps). Neighbouring pooling units take input from patches that are shifted by more than one row or column, thereby reducing the dimension of the representation and creating an invariance to small shifts and distortions. Two or three stages of convolution, non-linearity and pooling are stacked, followed by more convolutional and fully-connected layers. Backpropagating gradients through a ConvNet is as simple as through a regular deep network, allowing all the weights in all the filter banks to be trained.

Deep neural networks exploit the property that many natural signals are compositional hierarchies, in which higher-level features are obtained by composing lower-level ones. In images, local combinations of edges form motifs, motifs assemble into parts, and parts form objects. Similar hierarchies exist in speech and text from sounds to phones, phonemes, syllables, words and sentences. The pooling allows representations to vary very little when elements in the previous layer vary in position and appearance.

The convolutional and pooling layers in ConvNets are directly inspired by the classic notions of simple cells and complex cells in visual neuroscience[43], and the overall architecture is reminiscent of the LGN–V1–V2–V4–IT hierarchy in the visual cortex ventral pathway[44]. When ConvNet models and monkeys are shown the same picture, the activations of high-level units in the ConvNet explains half of the variance of random sets of 160 neurons in the monkey's inferotemporal cortex[45]. ConvNets have their roots in the neocognitron[46], the architecture of which was somewhat similar, but did not have an end-to-end supervised-learning algorithm such as backpropagation. A primitive 1D ConvNet called a time-delay neural net was used for the recognition of phonemes and simple words[47,48].

There have been numerous applications of convolutional networks going back to the early 1990s, starting with time-delay neural networks for speech recognition[47] and document reading[42]. The document reading system used a ConvNet trained jointly with a probabilistic model that implemented language constraints. By the late 1990s this system was reading over 10% of all the cheques in the United States. A number of ConvNet-based optical character recognition and handwriting recognition systems were later deployed by Microsoft[49]. ConvNets were also experimented with in the early 1990s for object detection in natural images, including faces and hands[50,51], and for face recognition[52].

Image understanding with deep convolutional networks

Since the early 2000s, ConvNets have been applied with great success to the detection, segmentation and recognition of objects and regions in images. These were all tasks in which labelled data was relatively abundant, such as traffic sign recognition[53], the segmentation of biological images[54] particularly for connectomics[55], and the detection of faces, text, pedestrians and human bodies in natural images[36,50,51,56–58]. A major recent practical success of ConvNets is face recognition[59].

Importantly, images can be labelled at the pixel level, which will have applications in technology, including autonomous mobile robots and

© 2015 Macmillan Publishers Limited. All rights reserved

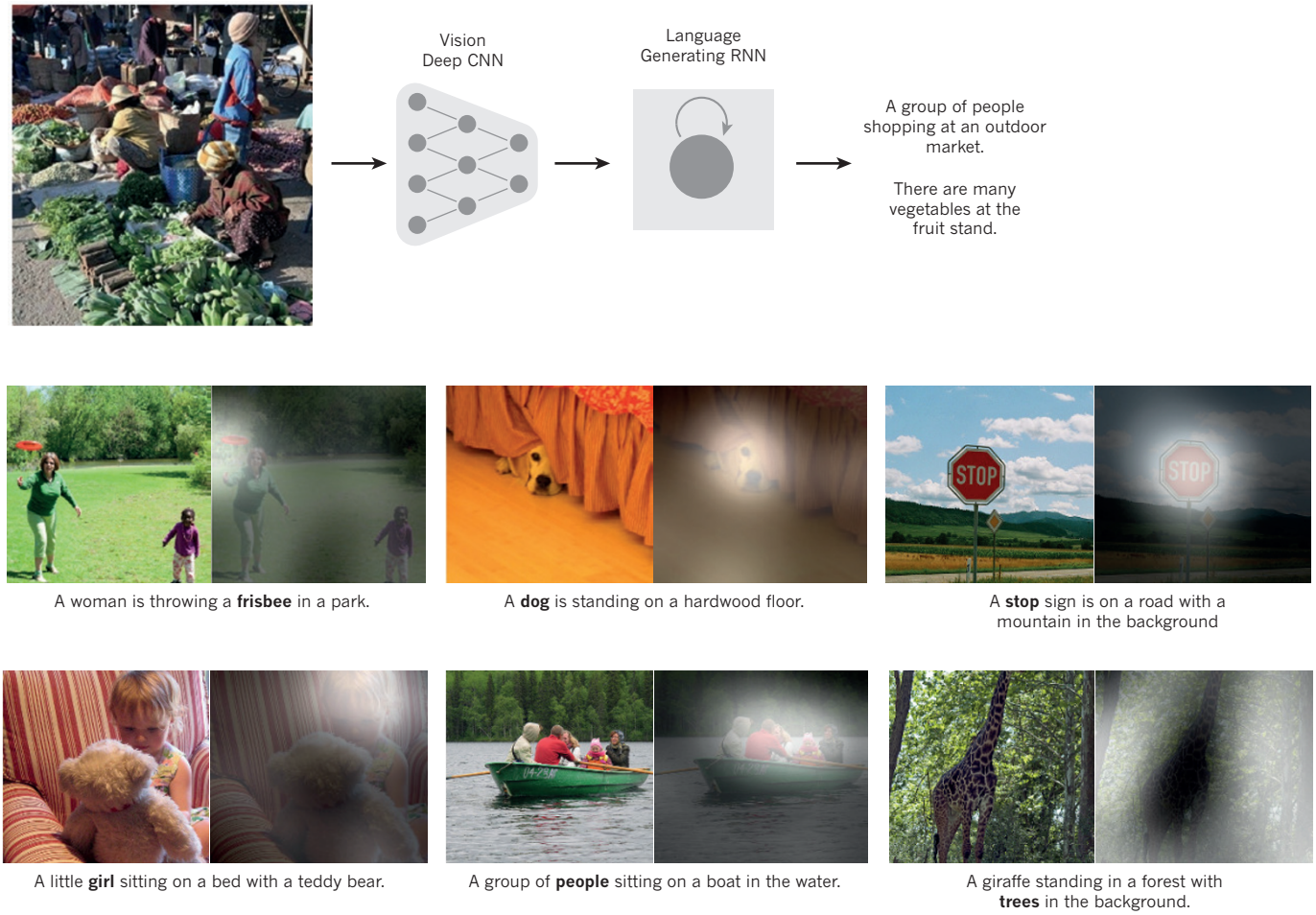

Figure 3 | From image to text. Captions generated by a recurrent neural network (RNN) taking, as extra input, the representation extracted by a deep convolution neural network (CNN) from a test image, with the RNN trained to 'translate' high-level representations of images into captions (top). Reproduced with permission from ref. 102. When the RNN is given the ability to focus its attention on a different location in the input image (middle and bottom; the lighter patches were given more attention) as it generates each word (bold), we found[86] that it exploits this to achieve better 'translation' of images into captions.

self-driving cars[60,61]. Companies such as Mobileye and NVIDIA are using such ConvNet-based methods in their upcoming vision systems for cars. Other applications gaining importance involve natural language understanding[14] and speech recognition[7].

Despite these successes, ConvNets were largely forsaken by the mainstream computer-vision and machine-learning communities until the ImageNet competition in 2012. When deep convolutional networks were applied to a data set of about a million images from the web that contained 1,000 different classes, they achieved spectacular results, almost halving the error rates of the best competing approaches[1]. This success came from the efficient use of GPUs, ReLUs, a new regularization technique called dropout[62], and techniques to generate more training examples by deforming the existing ones. This success has brought about a revolution in computer vision; ConvNets are now the dominant approach for almost all recognition and detection tasks[4,58,59,63–65] and approach human performance on some tasks. A recent stunning demonstration combines ConvNets and recurrent net modules for the generation of image captions (Fig. 3).

Recent ConvNet architectures have 10 to 20 layers of ReLUs, hundreds of millions of weights, and billions of connections between units. Whereas training such large networks could have taken weeks only two years ago, progress in hardware, software and algorithm parallelization have reduced training times to a few hours.

The performance of ConvNet-based vision systems has caused most major technology companies, including Google, Facebook, Microsoft, IBM, Yahoo!, Twitter and Adobe, as well as a quickly growing number of start-ups to initiate research and development projects and to deploy ConvNet-based image understanding products and services.

ConvNets are easily amenable to efficient hardware implementations in chips or field-programmable gate arrays[66,67]. A number of companies such as NVIDIA, Mobileye, Intel, Qualcomm and Samsung are developing ConvNet chips to enable real-time vision applications in smartphones, cameras, robots and self-driving cars.

Distributed representations and language processing

Deep-learning theory shows that deep nets have two different exponential advantages over classic learning algorithms that do not use distributed representations[21]. Both of these advantages arise from the power of composition and depend on the underlying data-generating distribution having an appropriate componential structure[40]. First, learning distributed representations enable generalization to new combinations of the values of learned features beyond those seen during training (for example, 2^n combinations are possible with n binary features)[68,69]. Second, composing layers of representation in a deep net brings the potential for another exponential advantage[70] (exponential in the depth).

The hidden layers of a multilayer neural network learn to represent the network's inputs in a way that makes it easy to predict the target outputs. This is nicely demonstrated by training a multilayer neural network to predict the next word in a sequence from a local

© 2015 Macmillan Publishers Limited. All rights reserved

context of earlier words[71]. Each word in the context is presented to the network as a one-of-N vector, that is, one component has a value of 1 and the rest are 0. In the first layer, each word creates a different pattern of activations, or word vectors (Fig. 4). In a language model, the other layers of the network learn to convert the input word vectors into an output word vector for the predicted next word, which can be used to predict the probability for any word in the vocabulary to appear as the next word. The network learns word vectors that contain many active components each of which can be interpreted as a separate feature of the word, as was first demonstrated[27] in the context of learning distributed representations for symbols. These semantic features were not explicitly present in the input. They were discovered by the learning procedure as a good way of factorizing the structured relationships between the input and output symbols into multiple 'micro-rules'. Learning word vectors turned out to also work very well when the word sequences come from a large corpus of real text and the individual micro-rules are unreliable[71]. When trained to predict the next word in a news story, for example, the learned word vectors for Tuesday and Wednesday are very similar, as are the word vectors for Sweden and Norway. Such representations are called distributed representations because their elements (the features) are not mutually exclusive and their many configurations correspond to the variations seen in the observed data. These word vectors are composed of learned features that were not determined ahead of time by experts, but automatically discovered by the neural network. Vector representations of words learned from text are now very widely used in natural language applications[14,17,72–76].

The issue of representation lies at the heart of the debate between the logic-inspired and the neural-network-inspired paradigms for cognition. In the logic-inspired paradigm, an instance of a symbol is something for which the only property is that it is either identical or non-identical to other symbol instances. It has no internal structure that is relevant to its use; and to reason with symbols, they must be bound to the variables in judiciously chosen rules of inference. By contrast, neural networks just use big activity vectors, big weight matrices and scalar non-linearities to perform the type of fast 'intuitive' inference that underpins effortless commonsense reasoning.

Before the introduction of neural language models[71], the standard approach to statistical modelling of language did not exploit distributed representations: it was based on counting frequencies of occurrences of short symbol sequences of length up to N (called N-grams). The number of possible N-grams is on the order of V^N, where V is the vocabulary size, so taking into account a context of more than a handful of words would require very large training corpora. N-grams treat each word as an atomic unit, so they cannot generalize across semantically related sequences of words, whereas neural language models can because they associate each word with a vector of real valued features, and semantically related words end up close to each other in that vector space (Fig. 4).

Recurrent neural networks

When backpropagation was first introduced, its most exciting use was for training recurrent neural networks (RNNs). For tasks that involve sequential inputs, such as speech and language, it is often better to use RNNs (Fig. 5). RNNs process an input sequence one element at a time, maintaining in their hidden units a 'state vector' that implicitly contains information about the history of all the past elements of the sequence. When we consider the outputs of the hidden units at different discrete time steps as if they were the outputs of different neurons in a deep multilayer network (Fig. 5, right), it becomes clear how we can apply backpropagation to train RNNs.

RNNs are very powerful dynamic systems, but training them has proved to be problematic because the backpropagated gradients either grow or shrink at each time step, so over many time steps they typically explode or vanish[77,78].

Thanks to advances in their architecture[79,80] and ways of training them[81,82], RNNs have been found to be very good at predicting the next character in the text[83] or the next word in a sequence[75], but they can also be used for more complex tasks. For example, after reading an English sentence one word at a time, an English 'encoder' network can be trained so that the final state vector of its hidden units is a good representation of the thought expressed by the sentence. This thought vector can then be used as the initial hidden state of (or as extra input to) a jointly trained French 'decoder' network, which outputs a probability distribution for the first word of the French translation. If a particular first word is chosen from this distribution and provided as input to the decoder network it will then output a probability distribution for the second word of the translation and so on until a full stop is chosen[17,72,76]. Overall, this process generates sequences of French words according to a probability distribution that depends on the English sentence. This rather naive way of performing machine translation has quickly become competitive with the state-of-the-art, and this raises serious doubts about whether understanding a sentence requires anything like the internal symbolic expressions that are manipulated by using inference rules. It is more compatible with the view that everyday reasoning involves many simultaneous analogies

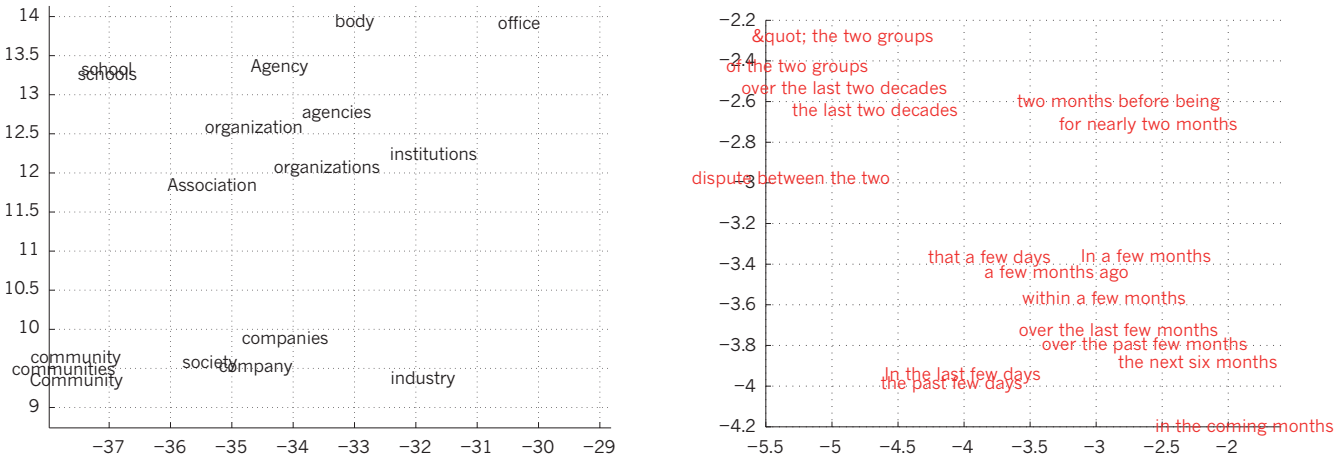

Figure 4 | Visualizing the learned word vectors. On the left is an illustration of word representations learned for modelling language, non-linearly projected to 2D for visualization using the t-SNE algorithm[103]. On the right is a 2D representation of phrases learned by an English-to-French encoder–decoder recurrent neural network[75]. One can observe that semantically similar words or sequences of words are mapped to nearby representations. The distributed representations of words are obtained by using backpropagation to jointly learn a representation for each word and a function that predicts a target quantity such as the next word in a sequence (for language modelling) or a whole sequence of translated words (for machine translation)[18,75].

© 2015 Macmillan Publishers Limited. All rights reserved

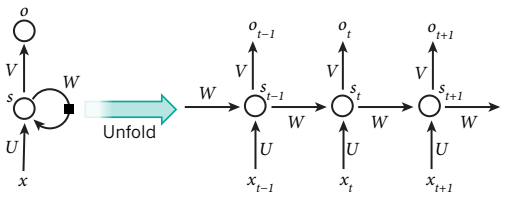

Figure 5 | A recurrent neural network and the unfolding in time of the computation involved in its forward computation. The artificial neurons (for example, hidden units grouped under node s with values s_t at time t) get inputs from other neurons at previous time steps (this is represented with the black square, representing a delay of one time step, on the left). In this way, a recurrent neural network can map an input sequence with elements x_t into an output sequence with elements o_t, with each o_t depending on all the previous $x_{t'}$ (for $t' \leq t$). The same parameters (matrices U, V, W) are used at each time step. Many other architectures are possible, including a variant in which the network can generate a sequence of outputs (for example, words), each of which is used as inputs for the next time step. The backpropagation algorithm (Fig. 1) can be directly applied to the computational graph of the unfolded network on the right, to compute the derivative of a total error (for example, the log-probability of generating the right sequence of outputs) with respect to all the states s_t and all the parameters.

that each contribute plausibility to a conclusion[84,85].

Instead of translating the meaning of a French sentence into an English sentence, one can learn to 'translate' the meaning of an image into an English sentence (Fig. 3). The encoder here is a deep ConvNet that converts the pixels into an activity vector in its last hidden layer. The decoder is an RNN similar to the ones used for machine translation and neural language modelling. There has been a surge of interest in such systems recently (see examples mentioned in ref. 86).

RNNs, once unfolded in time (Fig. 5), can be seen as very deep feedforward networks in which all the layers share the same weights. Although their main purpose is to learn long-term dependencies, theoretical and empirical evidence shows that it is difficult to learn to store information for very long[78].

To correct for that, one idea is to augment the network with an explicit memory. The first proposal of this kind is the long short-term memory (LSTM) networks that use special hidden units, the natural behaviour of which is to remember inputs for a long time[79]. A special unit called the memory cell acts like an accumulator or a gated leaky neuron: it has a connection to itself at the next time step that has a weight of one, so it copies its own real-valued state and accumulates the external signal, but this self-connection is multiplicatively gated by another unit that learns to decide when to clear the content of the memory.

LSTM networks have subsequently proved to be more effective than conventional RNNs, especially when they have several layers for each time step[87], enabling an entire speech recognition system that goes all the way from acoustics to the sequence of characters in the transcription. LSTM networks or related forms of gated units are also currently used for the encoder and decoder networks that perform so well at machine translation[17,72,76].

Over the past year, several authors have made different proposals to augment RNNs with a memory module. Proposals include the Neural Turing Machine in which the network is augmented by a 'tape-like' memory that the RNN can choose to read from or write to[88], and memory networks, in which a regular network is augmented by a kind of associative memory[89]. Memory networks have yielded excellent performance on standard question-answering benchmarks. The memory is used to remember the story about which the network is later asked to answer questions.

Beyond simple memorization, neural Turing machines and memory networks are being used for tasks that would normally require reasoning and symbol manipulation. Neural Turing machines can be taught 'algorithms'. Among other things, they can learn to output a sorted list of symbols when their input consists of an unsorted sequence in which each symbol is accompanied by a real value that indicates its priority in the list[88]. Memory networks can be trained to keep track of the state of the world in a setting similar to a text adventure game and after reading a story, they can answer questions that require complex inference[90]. In one test example, the network is shown a 15-sentence version of the *The Lord of the Rings* and correctly answers questions such as "where is Frodo now?"[89].

The future of deep learning

Unsupervised learning[91–98] had a catalytic effect in reviving interest in deep learning, but has since been overshadowed by the successes of purely supervised learning. Although we have not focused on it in this Review, we expect unsupervised learning to become far more important in the longer term. Human and animal learning is largely unsupervised: we discover the structure of the world by observing it, not by being told the name of every object.

Human vision is an active process that sequentially samples the optic array in an intelligent, task-specific way using a small, high-resolution fovea with a large, low-resolution surround. We expect much of the future progress in vision to come from systems that are trained end-to-end and combine ConvNets with RNNs that use reinforcement learning to decide where to look. Systems combining deep learning and reinforcement learning are in their infancy, but they already outperform passive vision systems[99] at classification tasks and produce impressive results in learning to play many different video games[100].

Natural language understanding is another area in which deep learning is poised to make a large impact over the next few years. We expect systems that use RNNs to understand sentences or whole documents will become much better when they learn strategies for selectively attending to one part at a time[76,86].

Ultimately, major progress in artificial intelligence will come about through systems that combine representation learning with complex reasoning. Although deep learning and simple reasoning have been used for speech and handwriting recognition for a long time, new paradigms are needed to replace rule-based manipulation of symbolic expressions by operations on large vectors[101]. ■

Received 25 February; accepted 1 May 2015.

1. Krizhevsky, A., Sutskever, I. & Hinton, G. ImageNet classification with deep convolutional neural networks. In *Proc. Advances in Neural Information Processing Systems 25* 1090–1098 (2012).
This report was a breakthrough that used convolutional nets to almost halve the error rate for object recognition, and precipitated the rapid adoption of deep learning by the computer vision community.
2. Farabet, C., Couprie, C., Najman, L. & LeCun, Y. Learning hierarchical features for scene labeling. *IEEE Trans. Pattern Anal. Mach. Intell.* **35,** 1915–1929 (2013).
3. Tompson, J., Jain, A., LeCun, Y. & Bregler, C. Joint training of a convolutional network and a graphical model for human pose estimation. In *Proc. Advances in Neural Information Processing Systems 27* 1799–1807 (2014).
4. Szegedy, C. *et al.* Going deeper with convolutions. Preprint at http://arxiv.org/abs/1409.4842 (2014).
5. Mikolov, T., Deoras, A., Povey, D., Burget, L. & Cernocky, J. Strategies for training large scale neural network language models. In *Proc. Automatic Speech Recognition and Understanding* 196–201 (2011).
6. Hinton, G. *et al.* Deep neural networks for acoustic modeling in speech recognition. *IEEE Signal Processing Magazine* **29,** 82–97 (2012).
This joint paper from the major speech recognition laboratories, summarizing the breakthrough achieved with deep learning on the task of phonetic classification for automatic speech recognition, was the first major industrial application of deep learning.
7. Sainath, T., Mohamed, A.-R., Kingsbury, B. & Ramabhadran, B. Deep convolutional neural networks for LVCSR. In *Proc. Acoustics, Speech and Signal Processing* 8614–8618 (2013).
8. Ma, J., Sheridan, R. P., Liaw, A., Dahl, G. E. & Svetnik, V. Deep neural nets as a method for quantitative structure-activity relationships. *J. Chem. Inf. Model.* **55,** 263–274 (2015).
9. Ciodaro, T., Deva, D., de Seixas, J. & Damazio, D. Online particle detection with neural networks based on topological calorimetry information. *J. Phys. Conf. Series* **368,** 012030 (2012).
10. Kaggle. Higgs boson machine learning challenge. *Kaggle* https://www.kaggle.com/c/higgs-boson (2014).
11. Helmstaedter, M. *et al.* Connectomic reconstruction of the inner plexiform layer in the mouse retina. *Nature* **500,** 168–174 (2013).

© 2015 Macmillan Publishers Limited. All rights reserved

12. Leung, M. K., Xiong, H. Y., Lee, L. J. & Frey, B. J. Deep learning of the tissue-regulated splicing code. *Bioinformatics* **30,** i121–i129 (2014).
13. Xiong, H. Y. *et al*. The human splicing code reveals new insights into the genetic determinants of disease. *Science* **347,** 6218 (2015).
14. Collobert, R., *et al*. Natural language processing (almost) from scratch. *J. Mach. Learn. Res.* **12,** 2493–2537 (2011).
15. Bordes, A., Chopra, S. & Weston, J. Question answering with subgraph embeddings. In *Proc. Empirical Methods in Natural Language Processing* http://arxiv.org/abs/1406.3676v3 (2014).
16. Jean, S., Cho, K., Memisevic, R. & Bengio, Y. On using very large target vocabulary for neural machine translation. In *Proc. ACL-IJCNLP* http://arxiv.org/abs/1412.2007 (2015).
17. Sutskever, I. Vinyals, O. & Le. Q. V. Sequence to sequence learning with neural networks. In *Proc. Advances in Neural Information Processing Systems 27* 3104–3112 (2014).
This paper showed state-of-the-art machine translation results with the architecture introduced in ref. 72, with a recurrent network trained to read a sentence in one language, produce a semantic representation of its meaning, and generate a translation in another language.
18. Bottou, L. & Bousquet, O. The tradeoffs of large scale learning. In *Proc. Advances in Neural Information Processing Systems 20* 161–168 (2007).
19. Duda, R. O. & Hart, P. E. *Pattern Classification and Scene Analysis* (Wiley, 1973).
20. Schölkopf, B. & Smola, A. *Learning with Kernels* (MIT Press, 2002).
21. Bengio, Y., Delalleau, O. & Le Roux, N. The curse of highly variable functions for local kernel machines. In *Proc. Advances in Neural Information Processing Systems 18* 107–114 (2005).
22. Selfridge, O. G. Pandemonium: a paradigm for learning in mechanisation of thought processes. In *Proc. Symposium on Mechanisation of Thought Processes* 513–526 (1958).
23. Rosenblatt, F. *The Perceptron — A Perceiving and Recognizing Automaton*. Tech. Rep. 85-460-1 (Cornell Aeronautical Laboratory, 1957).
24. Werbos, P. *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. PhD thesis, Harvard Univ. (1974).
25. Parker, D. B. *Learning Logic* Report TR–47 (MIT Press, 1985).
26. LeCun, Y. Une procédure d'apprentissage pour Réseau à seuil assymétrique in *Cognitiva 85: a la Frontière de l'Intelligence Artificielle, des Sciences de la Connaissance et des Neurosciences* [in French] 599–604 (1985).
27. Rumelhart, D. E., Hinton, G. E. & Williams, R. J. Learning representations by back-propagating errors. *Nature* **323,** 533–536 (1986).
28. Glorot, X., Bordes, A. & Bengio. Y. Deep sparse rectifier neural networks. In *Proc. 14th International Conference on Artificial Intelligence and Statistics* 315–323 (2011).
This paper showed that supervised training of very deep neural networks is much faster if the hidden layers are composed of ReLU.
29. Dauphin, Y. *et al*. Identifying and attacking the saddle point problem in high-dimensional non-convex optimization. In *Proc. Advances in Neural Information Processing Systems 27* 2933–2941 (2014).
30. Choromanska, A., Henaff, M., Mathieu, M., Arous, G. B. & LeCun, Y. The loss surface of multilayer networks. In *Proc. Conference on AI and Statistics* http://arxiv.org/abs/1412.0233 (2014).
31. Hinton, G. E. What kind of graphical model is the brain? In *Proc. 19th International Joint Conference on Artificial intelligence* 1765–1775 (2005).
32. Hinton, G. E., Osindero, S. & Teh, Y.-W. A fast learning algorithm for deep belief nets. *Neural Comp.* **18,** 1527–1554 (2006).
This paper introduced a novel and effective way of training very deep neural networks by pre-training one hidden layer at a time using the unsupervised learning procedure for restricted Boltzmann machines.
33. Bengio, Y., Lamblin, P., Popovici, D. & Larochelle, H. Greedy layer-wise training of deep networks. In *Proc. Advances in Neural Information Processing Systems 19* 153–160 (2006).
This report demonstrated that the unsupervised pre-training method introduced in ref. 32 significantly improves performance on test data and generalizes the method to other unsupervised representation-learning techniques, such as auto-encoders.
34. Ranzato, M., Poultney, C., Chopra, S. & LeCun, Y. Efficient learning of sparse representations with an energy-based model. In *Proc. Advances in Neural Information Processing Systems 19* 1137–1144 (2006).
35. Hinton, G. E. & Salakhutdinov, R. Reducing the dimensionality of data with neural networks. *Science* **313,** 504–507 (2006).
36. Sermanet, P., Kavukcuoglu, K., Chintala, S. & LeCun, Y. Pedestrian detection with unsupervised multi-stage feature learning. In *Proc. International Conference on Computer Vision and Pattern Recognition* http://arxiv.org/abs/1212.0142 (2013).
37. Raina, R., Madhavan, A. & Ng, A. Y. Large-scale deep unsupervised learning using graphics processors. In *Proc. 26th Annual International Conference on Machine Learning* 873–880 (2009).
38. Mohamed, A.-R., Dahl, G. E. & Hinton, G. Acoustic modeling using deep belief networks. *IEEE Trans. Audio Speech Lang. Process.* **20,** 14–22 (2012).
39. Dahl, G. E., Yu, D., Deng, L. & Acero, A. Context-dependent pre-trained deep neural networks for large vocabulary speech recognition. *IEEE Trans. Audio Speech Lang. Process.* **20,** 33–42 (2012).
40. Bengio, Y., Courville, A. & Vincent, P. Representation learning: a review and new perspectives. *IEEE Trans. Pattern Anal. Machine Intell.* **35,** 1798–1828 (2013).
41. LeCun, Y. *et al*. Handwritten digit recognition with a back-propagation network. In *Proc. Advances in Neural Information Processing Systems* 396–404 (1990).
This is the first paper on convolutional networks trained by backpropagation for the task of classifying low-resolution images of handwritten digits.
42. LeCun, Y., Bottou, L., Bengio, Y. & Haffner, P. Gradient-based learning applied to document recognition. *Proc. IEEE* **86,** 2278–2324 (1998).
This overview paper on the principles of end-to-end training of modular systems such as deep neural networks using gradient-based optimization showed how neural networks (and in particular convolutional nets) can be combined with search or inference mechanisms to model complex outputs that are interdependent, such as sequences of characters associated with the content of a document.
43. Hubel, D. H. & Wiesel, T. N. Receptive fields, binocular interaction, and functional architecture in the cat's visual cortex. *J. Physiol.* **160,** 106–154 (1962).
44. Felleman, D. J. & Essen, D. C. V. Distributed hierarchical processing in the primate cerebral cortex. *Cereb. Cortex* **1,** 1–47 (1991).
45. Cadieu, C. F. *et al*. Deep neural networks rival the representation of primate it cortex for core visual object recognition. *PLoS Comp. Biol.* **10,** e1003963 (2014).
46. Fukushima, K. & Miyake, S. Neocognitron: a new algorithm for pattern recognition tolerant of deformations and shifts in position. *Pattern Recognition* **15,** 455–469 (1982).
47. Waibel, A., Hanazawa, T., Hinton, G. E., Shikano, K. & Lang, K. Phoneme recognition using time-delay neural networks. *IEEE Trans. Acoustics Speech Signal Process.* **37,** 328–339 (1989).
48. Bottou, L., Fogelman-Soulié, F., Blanchet, P. & Lienard, J. Experiments with time delay networks and dynamic time warping for speaker independent isolated digit recognition. In *Proc. EuroSpeech 89* 537–540 (1989).
49. Simard, D., Steinkraus, P. Y. & Platt, J. C. Best practices for convolutional neural networks. In *Proc. Document Analysis and Recognition* 958–963 (2003).
50. Vaillant, R., Monrocq, C. & LeCun, Y. Original approach for the localisation of objects in images. In *Proc. Vision, Image, and Signal Processing* **141,** 245–250 (1994).
51. Nowlan, S. & Platt, J. in *Neural Information Processing Systems* 901–908 (1995).
52. Lawrence, S., Giles, C. L., Tsoi, A. C. & Back, A. D. Face recognition: a convolutional neural-network approach. *IEEE Trans. Neural Networks* **8,** 98–113 (1997).
53. Ciresan, D., Meier, U. Masci, J. & Schmidhuber, J. Multi-column deep neural network for traffic sign classification. *Neural Networks* **32,** 333–338 (2012).
54. Ning, F. *et al*. Toward automatic phenotyping of developing embryos from videos. *IEEE Trans. Image Process.* **14,** 1360–1371 (2005).
55. Turaga, S. C. *et al.* Convolutional networks can learn to generate affinity graphs for image segmentation. *Neural Comput.* **22,** 511–538 (2010).
56. Garcia, C. & Delakis, M. Convolutional face finder: a neural architecture for fast and robust face detection. *IEEE Trans. Pattern Anal. Machine Intell.* **26,** 1408–1423 (2004).
57. Osadchy, M., LeCun, Y. & Miller, M. Synergistic face detection and pose estimation with energy-based models. *J. Mach. Learn. Res.* **8,** 1197–1215 (2007).
58. Tompson, J., Goroshin, R. R., Jain, A., LeCun, Y. Y. & Bregler, C. C. Efficient object localization using convolutional networks. In *Proc. Conference on Computer Vision and Pattern Recognition* http://arxiv.org/abs/1411.4280 (2014).
59. Taigman, Y., Yang, M., Ranzato, M. & Wolf, L. Deepface: closing the gap to human-level performance in face verification. In *Proc. Conference on Computer Vision and Pattern Recognition* 1701–1708 (2014).
60. Hadsell, R. *et al.* Learning long-range vision for autonomous off-road driving. *J. Field Robot.* **26,** 120–144 (2009).
61. Farabet, C., Couprie, C., Najman, L. & LeCun, Y. Scene parsing with multiscale feature learning, purity trees, and optimal covers. In *Proc. International Conference on Machine Learning* http://arxiv.org/abs/1202.2160 (2012).
62. Srivastava, N., Hinton, G., Krizhevsky, A., Sutskever, I. & Salakhutdinov, R. Dropout: a simple way to prevent neural networks from overfitting. *J. Machine Learning Res.* **15,** 1929–1958 (2014).
63. Sermanet, P. *et al*. Overfeat: integrated recognition, localization and detection using convolutional networks. In *Proc. International Conference on Learning Representations* http://arxiv.org/abs/1312.6229 (2014).
64. Girshick, R., Donahue, J., Darrell, T. & Malik, J. Rich feature hierarchies for accurate object detection and semantic segmentation. In *Proc. Conference on Computer Vision and Pattern Recognition* 580–587 (2014).
65. Simonyan, K. & Zisserman, A. Very deep convolutional networks for large-scale image recognition. In *Proc. International Conference on Learning Representations* http://arxiv.org/abs/1409.1556 (2014).
66. Boser, B., Sackinger, E., Bromley, J., LeCun, Y. & Jackel, L. An analog neural network processor with programmable topology. *J. Solid State Circuits* **26,** 2017–2025 (1991).
67. Farabet, C. *et al.* Large-scale FPGA-based convolutional networks. In *Scaling up Machine Learning: Parallel and Distributed Approaches* (eds Bekkerman, R., Bilenko, M. & Langford, J.) 399–419 (Cambridge Univ. Press, 2011).
68. Bengio, Y. *Learning Deep Architectures for AI* (Now, 2009).
69. Montufar, G. & Morton, J. When does a mixture of products contain a product of mixtures? *J. Discrete Math.* **29,** 321–347 (2014).
70. Montufar, G. F., Pascanu, R., Cho, K. & Bengio, Y. On the number of linear regions of deep neural networks. In *Proc. Advances in Neural Information Processing Systems 27* 2924–2932 (2014).
71. Bengio, Y., Ducharme, R. & Vincent, P. A neural probabilistic language model. In *Proc. Advances in Neural Information Processing Systems 13* 932–938 (2001).
This paper introduced neural language models, which learn to convert a word symbol into a word vector or word embedding composed of learned semantic features in order to predict the next word in a sequence.
72. Cho, K. *et al.* Learning phrase representations using RNN encoder-decoder

© 2015 Macmillan Publishers Limited. All rights reserved

for statistical machine translation. In *Proc. Conference on Empirical Methods in Natural Language Processing* 1724–1734 (2014).
73. Schwenk, H. Continuous space language models. *Computer Speech Lang.* **21,** 492–518 (2007).
74. Socher, R., Lin, C. C-Y., Manning, C. & Ng, A. Y. Parsing natural scenes and natural language with recursive neural networks. In *Proc. International Conference on Machine Learning* 129–136 (2011).
75. Mikolov, T., Sutskever, I., Chen, K., Corrado, G. & Dean, J. Distributed representations of words and phrases and their compositionality. In *Proc. Advances in Neural Information Processing Systems 26* 3111–3119 (2013).
76. Bahdanau, D., Cho, K. & Bengio, Y. Neural machine translation by jointly learning to align and translate. In *Proc. International Conference on Learning Representations* http://arxiv.org/abs/1409.0473 (2015).
77. Hochreiter, S. Untersuchungen zu dynamischen neuronalen Netzen [in German] Diploma thesis, T.U. Münich (1991).
78. Bengio, Y., Simard, P. & Frasconi, P. Learning long-term dependencies with gradient descent is difficult. *IEEE Trans. Neural Networks* **5,** 157–166 (1994).
79. Hochreiter, S. & Schmidhuber, J. Long short-term memory. *Neural Comput.* **9,** 1735–1780 (1997).
This paper introduced LSTM recurrent networks, which have become a crucial ingredient in recent advances with recurrent networks because they are good at learning long-range dependencies.
80. ElHihi, S. & Bengio, Y. Hierarchical recurrent neural networks for long-term dependencies. In *Proc. Advances in Neural Information Processing Systems 8* http://papers.nips.cc/paper/1102-hierarchical-recurrent-neural-networks-for-long-term-dependencies (1995).
81. Sutskever, I. *Training Recurrent Neural Networks*. PhD thesis, Univ. Toronto (2012).
82. Pascanu, R., Mikolov, T. & Bengio, Y. On the difficulty of training recurrent neural networks. In *Proc. 30th International Conference on Machine Learning* 1310–1318 (2013).
83. Sutskever, I., Martens, J. & Hinton, G. E. Generating text with recurrent neural networks. In *Proc. 28th International Conference on Machine Learning* 1017–1024 (2011).
84. Lakoff, G. & Johnson, M. *Metaphors We Live By* (Univ. Chicago Press, 2008).
85. Rogers, T. T. & McClelland, J. L. *Semantic Cognition: A Parallel Distributed Processing Approach* (MIT Press, 2004).
86. Xu, K. *et al*. Show, attend and tell: Neural image caption generation with visual attention. In *Proc. International Conference on Learning Representations* http://arxiv.org/abs/1502.03044 (2015).
87. Graves, A., Mohamed, A.-R. & Hinton, G. Speech recognition with deep recurrent neural networks. In *Proc. International Conference on Acoustics, Speech and Signal Processing* 6645–6649 (2013).
88. Graves, A., Wayne, G. & Danihelka, I. Neural Turing machines. http://arxiv.org/abs/1410.5401 (2014).
89. Weston, J. Chopra, S. & Bordes, A. Memory networks. http://arxiv.org/abs/1410.3916 (2014).
90. Weston, J., Bordes, A., Chopra, S. & Mikolov, T. Towards AI-complete question answering: a set of prerequisite toy tasks. http://arxiv.org/abs/1502.05698 (2015).
91. Hinton, G. E., Dayan, P., Frey, B. J. & Neal, R. M. The wake-sleep algorithm for unsupervised neural networks. *Science* **268,** 1558–1161 (1995).
92. Salakhutdinov, R. & Hinton, G. Deep Boltzmann machines. In *Proc. International Conference on Artificial Intelligence and Statistics* 448–455 (2009).
93. Vincent, P., Larochelle, H., Bengio, Y. & Manzagol, P.-A. Extracting and composing robust features with denoising autoencoders. In *Proc. 25th International Conference on Machine Learning* 1096–1103 (2008).
94. Kavukcuoglu, K. *et al*. Learning convolutional feature hierarchies for visual recognition. In *Proc. Advances in Neural Information Processing Systems 23* 1090–1098 (2010).
95. Gregor, K. & LeCun, Y. Learning fast approximations of sparse coding. In *Proc. International Conference on Machine Learning* 399–406 (2010).
96. Ranzato, M., Mnih, V., Susskind, J. M. & Hinton, G. E. Modeling natural images using gated MRFs. *IEEE Trans. Pattern Anal. Machine Intell.* **35,** 2206–2222 (2013).
97. Bengio, Y., Thibodeau-Laufer, E., Alain, G. & Yosinski, J. Deep generative stochastic networks trainable by backprop. In *Proc. 31st International Conference on Machine Learning* 226–234 (2014).
98. Kingma, D., Rezende, D., Mohamed, S. & Welling, M. Semi-supervised learning with deep generative models. In *Proc. Advances in Neural Information Processing Systems 27* 3581–3589 (2014).
99. Ba, J., Mnih, V. & Kavukcuoglu, K. Multiple object recognition with visual attention. In *Proc. International Conference on Learning Representations* http://arxiv.org/abs/1412.7755 (2014).
100. Mnih, V. *et al*. Human-level control through deep reinforcement learning. *Nature* **518,** 529–533 (2015).
101. Bottou, L. From machine learning to machine reasoning. *Mach. Learn.* **94,** 133–149 (2014).
102. Vinyals, O., Toshev, A., Bengio, S. & Erhan, D. Show and tell: a neural image caption generator. In *Proc. International Conference on Machine Learning* http://arxiv.org/abs/1502.03044 (2014).
103. van der Maaten, L. & Hinton, G. E. Visualizing data using t-SNE. *J. Mach. Learn. Research* **9,** 2579–2605 (2008).

Acknowledgements The authors would like to thank the Natural Sciences and Engineering Research Council of Canada, the Canadian Institute For Advanced Research (CIFAR), the National Science Foundation and Office of Naval Research for support. Y.L. and Y.B. are CIFAR fellows.

Author Information Reprints and permissions information is available at www.nature.com/reprints. The authors declare no competing financial interests. Readers are welcome to comment on the online version of this paper at go.nature.com/7cjbaa. Correspondence should be addressed to Y.L. (yann@cs.nyu.edu).

© 2015 Macmillan Publishers Limited. All rights reserved